JavaScript

Basic Concepts of Dynamic Pages

language characteristics, BOM, DOM, Events, Fetch, Frameworks

Spolufinancováno Evropskou unií

MINISTERSTVO ŠKOLSTVÍ, MLÁDEŽE A TĚLOVÝCHOVY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

doc. Ing. Radek Burget, Ph.D.

burgetr@vut.cz

Ing. Jiří Hynek, Ph.D.

hynek@vut.cz

Motivation

- **understand client-side scripting**
  - **dynamic content**
    - generate page content based on conditions
    - e.g. current time, user data, browser version
  - **interactivity**
    - react to user events (clicks, mouse movement, input, …)
    - modify page elements and styles in real time
  - **browser control**
    - navigate browser history
    - open windows/tabs
    - communicate with the browser environment
  - **data fetching a sending**
    - dynamic data fetch, update content

Language

concepts

JavaScript

- high-level programming language for the web
- runs directly in the browser (client-side)
- **language characteristics:**
  - dynamic – types are determined at runtime
  - weakly typed – automatic type conversions
  - scripting language – executed directly without compilation
  - interpreted / JIT compiled – runs immediately in the browser
  - multi-paradigm – supports procedural, object-oriented and functional styles
  - prototype-based – objects inherit from other objects
  - first-class functions – functions can be passed and returned like values
  - event-driven – reacts to user and browser events

History

- **1995** – the language created (Netscape, Brendan Eich): Mocha → LiveScript → JavaScript
  - the final name chosen to leverage Java's popularity at the time
  - *Java and JavaScript are like car and carpet* (just a joke, no offense)
- **1997** – ECMAScript standard
- **2009** – Node.js (server-side JS)
- **2015** (ES6) – modern JavaScript (modules, classes, arrow functions, …)
- **Today** – most widely used web language

- **ECMAScript (ES) standard**
  - defines the language syntax, types, and core behavior
  - ensures compatibility across browsers and environments
  - browsers implement ECMAScript as JavaScript
  - new versions add modern features regularly

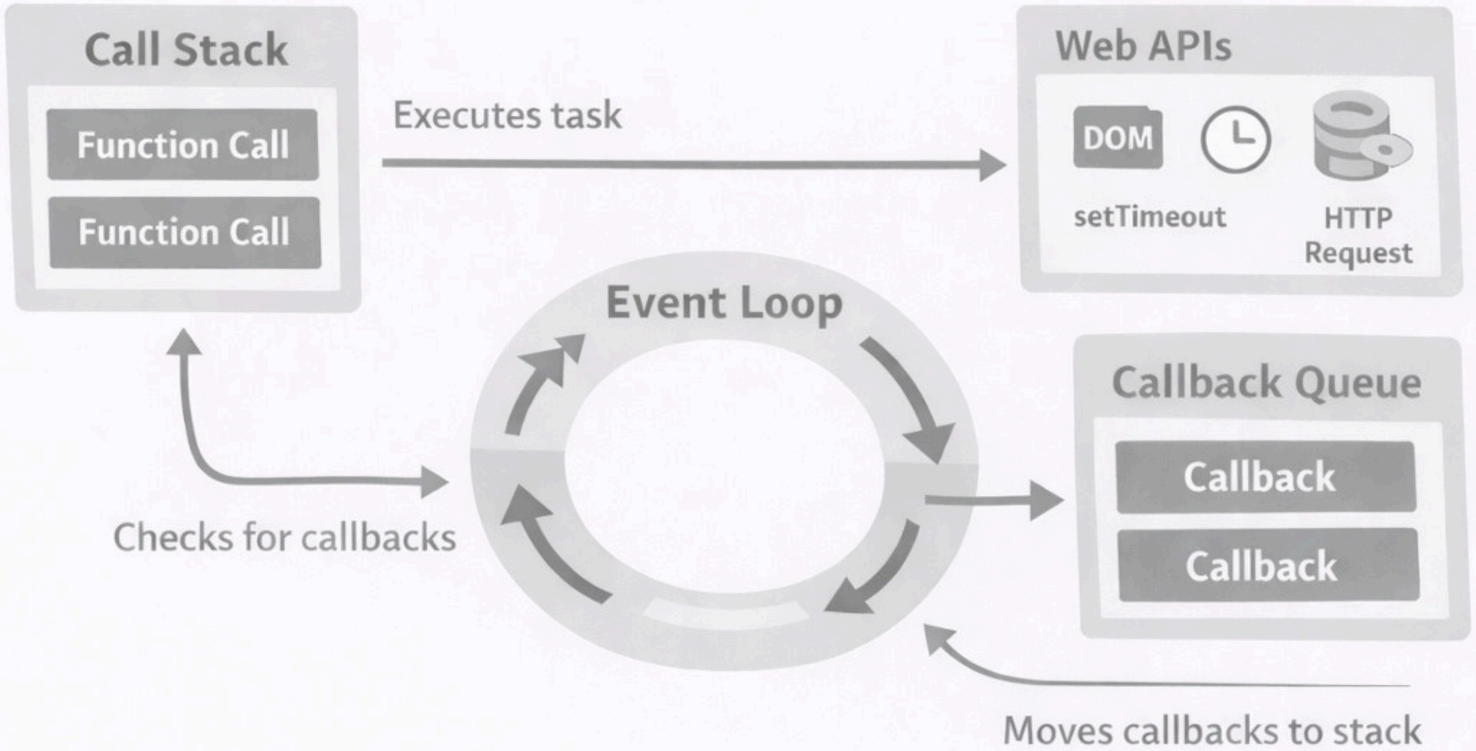
PLANET	DEATHS	HAS JAVASCRIPT
	0	NO
	0	NO
	120,318,942,654+	YES
	0	NO
	0	NO
	0	NO
	0	NO
	0	NO

Terminology

- **ECMAScript** – language specification (the standard), grammar rules
- **JavaScript** – ECMAScript implementation in web browsers
- **Runtime** – environment providing APIs and execution context (browser or Node.js)
  - Browser runtime – DOM, events, fetch, window
  - Node.js runtime – file system, network, process
- **Engine** – program that executes JavaScript code
  - parses and executes code
    - code is compiled during execution (JIT compilation – Just in Time) – optimization
  - V8 (Chrome, Node.js), SpiderMonkey (Firefox), JavaScriptCore/WebKit (Safari)

Event Loop

- JavaScript is single-threaded – only one task runs at a time
- long operations must be asynchronous
- the Event Loop coordinates task execution – it keeps the application responsive



Link JS to HTML

- **inline JavaScript**

```
<button onclick="alert('Hello!')">Click me</button>
```

- **internal script**

```
<script>
/* some JS code */
</script>
```

- **external script**

```
<script src="program.js" defer></script>
```

- `defer` – to run after HTML is parsed
- `async` – for parallel loading (executes as soon as ready)

Script Execution

- executed immediately when the browser encounters the `<script>` tag

```
<body>
  <p>Now the script is going to be executed.</p>
  <script>
    document.write("<p><em>The script is running.</em></p>");
  </script>
  <p>The script has been executed. We process the remaining document.</p>
</body>
```

```
Now the script is going to be executed.The script has been executed. We pr
```

Syntax

```javascript
// Characters array
const characters = [
  { name: "Jon Snow", house: "Stark", alive: true },
  { name: "Daenerys Targaryen", house: "Targaryen", alive: false },
  { name: "Tyrion Lannister", house: "Lannister", alive: true }
];

/* Function to greet characters */
function greetAlive(characters) {
  for(let char of characters){
    if(char.alive){
      console.log(`Hello, ${char.name} of House ${char.house}!`);
    } else {
      console.log(`${char.name} is no longer with us.`);
    }
  }
}

// Run function
greetAlive(characters);
```

Key Language Constructs

🟢 **Variables & Constants:** `let`, `const`, `var`

🔢 **Data Types:** number, string, boolean, null, undefined, object, array, function

➕➖ **Operators:** arithmetic, comparison, logical, assignment

🔄 **Control Flow:** `if…else`, `switch`, ternary `? :`

🔁 **Loops:** `for`, `for…of`, `for…in`, `while`, `do…while`

⚡ **Functions:** declarations, expressions, arrow functions, first-class functions

🗂️ **Objects & Arrays:** key/value pairs, methods, indexing

✨ **Template Literals:** `${}` for string interpolation

🖱️ **Events & DOM:** selecting, modifying, listening to HTML elements

⏱️ **Asynchronous:** `setTimeout`, `setInterval`, Promises, `async/await`

📦 **Modules:** `import` / `export`

BOM – Browser Object Model

window, navigator, location, history, screen, dialogs

Browser Object Model – window

- provides JavaScript access to browser-specific objects, outside the DOM
- enables interaction with the browser environment
- **window** – the global object representing the browser window
  - document – document content (DOM)
  - navigator – information about the browser
  - location – URL info and navigation
  - history – browser history manipulation
  - screen – screen resolution and display info
  - alert(), confirm(), prompt(), setTimeout() – simple dialog boxes
  - …

Hello World

```
/* Write "Hello, World!" to the console */
window.console.log("Hello, World!");

/* Write "Hello, World!" to the document */
window.document.write("Hello, World!");
```

- `window` is the global object, so we can omit it:

```
console.log("Hello, World!");
document.write("Hello, World!");
```

- vypíše se "Hello, World!" do konzole a do dokumentu (na stránku)

Navigator

- provides information about the browser and environment

```
document.write(navigator.appName);    // "Netscape" in modern browsers (legacy value)
document.write(navigator.appVersion); // detailed version info, e.g. "5.0 (Windows)"
document.write(navigator.userAgent);  // detailed user agent string
document.write(navigator.platform);   // platform info, e.g. "Win32", "MacIntel", "Linux x
document.write(navigator.language);   // browser language, e.g. "en-US"
document.write(navigator.onLine);     // boolean indicating if the browser is online
```

Location

- represents the current URL of the window

```
document.write(window.location.href);     // full URL
document.write(window.location.protocol); // "http:" or "https:"
document.write(window.location.host);     // "example.com:8000"
document.write(window.location.pathname); // "/page.html"
document.write(window.location.hash);     // "#section1"
```

Location: redirection

- change the URL and navigate to a different page

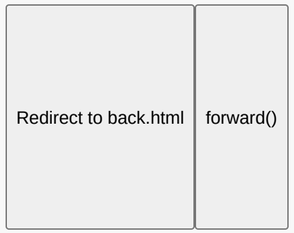
```
window.location.href = "https://www.example.com";
```

Redirect to example.com

History

- allows manipulation of the browser history stack

```
window.history.back();    // go back one page
window.history.forward(); // go forward one page
window.history.go(-2);   // go back two pages
window.history.go(1);    // go forward one page
```



- example: back.html and forward.html

Screen

- provides information about the user's screen and display capabilities

```
document.write(screen.width);        // total screen width
document.write(screen.height);       // total screen height
document.write(screen.availWidth);   // available width (excluding taskbar, etc.)
document.write(screen.availHeight);  // available height
document.write(screen.colorDepth);   // color depth in bits
document.write(screen.pixelDepth);   // pixel depth in bits
```

Dialogs

```
alert("Winter is coming!");

let ready = confirm("Are you ready to enter Westeros?");
if (ready) {
  let name = prompt("What is your name?");
  alert(`Welcome, ${name}!`);
} else {
  alert("Come back later!");
}
```

- dialogs are blocked in `iframe` – run example

- blocking and inflexible – freeze the page, cannot be styled or customized
- legacy and unreliable – often restricted or blocked (iframes, sandbox, modern browsers)

- alternative: custom modals or the modern `<dialog>` element – non-blocking, fully stylable, and more reliable

DOM – Document Object Model

document, dynamic elements modification

Document Object Model – document

- **window.document** – the root object representing the web page
- each HTML element becomes a node in a tree structure
- allows JavaScript to read, change, add, or remove content dynamically

```
Node
 ├─ Document
 ├─ Element
 │   ├─ HTMLElement
 │   │   ├─ HTMLDivElement
 │   │   ├─ HTMLParagraphElement
 │   │   ├─ HTMLButtonElement
 │   │   └─ HTMLInputElement
 │   └─ SVGElement
 └─ Text
```

MDN: Document, Element, HTMLElement

Query Elements

```
<h1 id="mainTitle">Game of Thrones</h1>
<ul>
  <li class="item">Tyrion</li>
  <li class="item">Arya</li>
  <li class="item">Sansa</li>
</ul>

<script>
  const header = document.getElementById("mainTitle");

  const itemsByClass = document.getElementsByClassName("item");

  const paragraphs = document.getElementsByTagName("p");

  const firstItem = document.querySelector(".item");

  const allItems = document.querySelectorAll(".item");

  console.log(header, itemsByClass, paragraphs, firstItem, allItems);
</script>
```

Example

Node, Element, HTMLElement

- **Node** – basic unit of the DOM tree, can be an element, text, comment, etc.
  - `parentNode`, `childNodes`, `nextSibling`, `previousSibling`, etc.

- **Element** – represents an HTML or XML element, inherits from Node
  - `tagName`, `id`, `className`, `attributes`, etc.

- **HTMLElement** – specific to HTML elements, inherits from Element
  - `innerHTML`, `innerText`, `style`, etc.

- other specific element types (HTMLDivElement, HTMLParagraphElement, etc.) inherit from HTMLElement and have additional properties/methods

Modifying Elements

```
<div id="box" class="container">
  <p class="text">Winter is coming!</p>
</div>

<script>
  const box = document.getElementById("box");
  const para = box.querySelector(".text");
  const btn = document.getElementById("btn");

  para.textContent = "Brace yourselves!"; // Change text content
  box.innerHTML += "<p>The North remembers.</p>"; // Change HTML

  box.style.backgroundColor = "lightblue"; // Change style
  para.style.color = "darkred";

  box.classList.add("highlight"); // Change classes
  para.classList.remove("text");
</script>
```

Example

Append and Remove Elements

```
<div id="box" class="container">
  <p id="note">Winter is coming!</p>
</div>

<script>
  const box = document.getElementById("box");
  const note = document.getElementById("note");

  // create new elements
  const note2 = document.createElement("p");
  note2.textContent = "Brace yourselves!";
  const img = document.createElement("img");
  img.src = "winter_is_coming.jpg";
  img.width = 200;

  // add and remove elements
  box.appendChild(note2);
  box.appendChild(img);
  note.remove();
</script>
```

Example

Events

types, listeners, propagation

Events

- Event – a signal that something happened in the browser
- triggered by user actions or system changes

Category	Events
🖱️ **Mouse**	click, dblclick, mousedown, mouseup, mousemove, mouseenter, mouseleave, mouseover, mouseout, contextmenu, wheel
⌨️ **Keyboard**	keydown, keyup, keypress
📱 **Touch**	touchstart, touchmove, touchend, touchcancel
🖊️ **Pointer**	pointerdown, pointerup, pointermove, pointerenter, pointerleave, pointerover, pointerout, pointercancel
📝 **Form**	input, change, submit, reset, focus, blur, focusin, focusout, select, invalid
🎬 **Media**	play, pause, ended, timeupdate, volumechange, loadeddata
🪟 **Window**	load, DOMContentLoaded, resize, scroll, beforeunload, unload, hashchange, popstate, error

- **pointer** – modern unified input

Event Listeners

- **inline HTML**

```html
<button onclick="doSomething()">Click</button>
```

- **DOM Element property**

```
button.onclick = doSomething; // only one event handler
```

- **addEventListener** (recommended)

```
addEventListener("click", doSomething);

addEventListener("click", (e) => {
  // another listener
});
```

Example

Event Propagation

- when an event occurs on an element, it travels through the DOM

1. **capturing phase** (trickling down) – event travels from the root down to the target element
2. **target phase** – event reaches the element that triggered it
3. **bubbling phase** (bubbling up) – event travels back up from target to root

- `event.stopPropagation()` – it stops propagation
- `event.preventDefault()` – it prevents default action

Example

DOM Ready

- `DOMContentLoaded` event – fired when the initial HTML document has been completely loaded and parsed, without waiting for stylesheets, images, and subframes to finish loading

```
<script>
  document.addEventListener("DOMContentLoaded", () => {
    // code to run when the DOM is ready
    const par = document.querySelector(".par");
    par.textContent = "The night is dark and full of terrors.";
  });
</script>

<!-- elements are located after the script -->
<p class="par">Winter is coming…</p>
```

Dynamic Data Fetch

fetch

Fetch API

- API for HTTP requests, replaces older XMLHttpRequest (XHR)
- Based on Promises
- Works with JSON, text, files, APIs

```
fetch(url, options)
  .then(response => {
    // handle HTTP response
    return response.json();
  })
  .then(data => {
    // work with data
    console.log(data);
  })
  .catch(error => {
    // handle network errors
    console.error(error);
  });
```

Fetch Request

- interface of the Fetch API which represents a resource request
- used to ask for data or send data (GET = retrieve, POST/PUT = send/update, DELETE)

```
fetch("/api/users", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "Authorization": "Bearer token123"
    },
    body: JSON.stringify({
      name: "Jon Snow",
      role: "Knights Watch"
    }),
    credentials: "include"
});
```

Fetch Response

- interface of the Fetch API represents the response to a request

- `response.ok` – true if status 200–299
- `response.status` – HTTP status code (200, 404, 500 …)
- `response.statusText` – status message ("OK", "Not Found")
- `response.type` – basic / cors / opaque

- `response.json()` – parses JSON data
- `response.text()` – gets response as text
- `response.blob()` – gets binary data (images, files)
- `response.formData()` – gets form submissions
- `response.arrayBuffer()` – gets low-level binary data

Async + await

- cleaner syntax for working with Promises
- try/catch for error handling

```
async function query() {
  try {
    const response = await fetch(url, options);

    if (!response.ok) {
      throw new Error("HTTP error");
    }

    const data = await response.json();
    console.log(data);

  } catch (error) {
    console.error(error);
  }
}

query().then(() => { … });
```

Example: then()

```
fetch("https://restcountries.com/v3.1/all?fields=name&fields=cca3")
  .then(response => {
    if (!response.ok) {
      throw new Error("HTTP error");
    }
    return response.json();
  })
  .then(countries => {
    console.log(countries);
  })
  .catch(error => {
    console.error(error);
  });
```

Example

Example: async + await

```
async function fetchCountries() {
  try {
    const response = await fetch("https://restcountries.com/v3.1/all?fields=name&fields=cc

    if (!response.ok) {
      throw new Error("HTTP error");
    }

    const countries = await response.json();
    console.log(countries);

  } catch (error) {
    console.error(error);
  }
}

fetchCountries();
```

Example

Example: Endless Page

Libraries & Frameworks

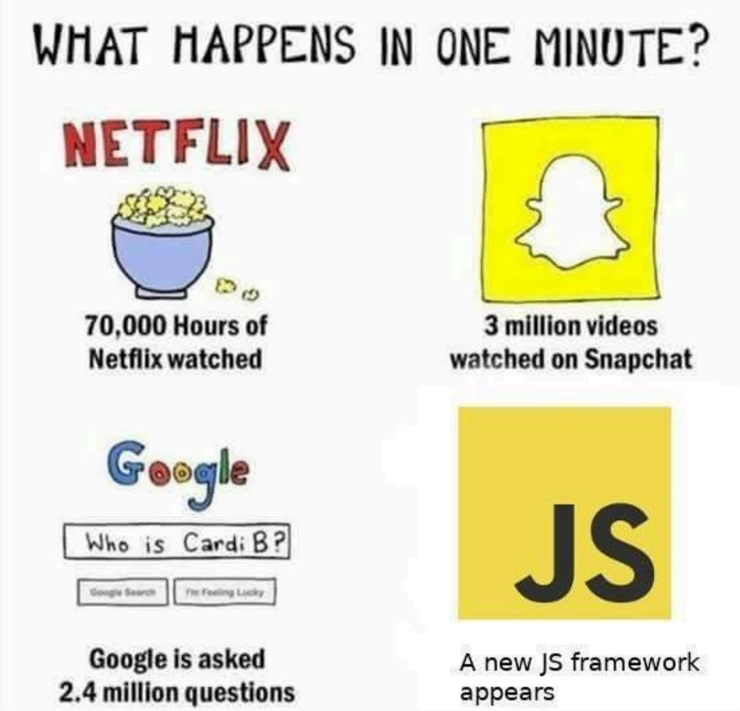
CDN, npm, JQuery, Lodash, D3.js, React, TypeScript

Motivation

- simplify development by providing reusable components and utilities
- abstract away browser inconsistencies and low-level details

- **libraries**: focused on specific tasks (e.g., DOM manipulation, AJAX)
  - e.g., jQuery, Lodash, D3.js

- **frameworks**: provide a structured approach to building applications
  - e.g., React, Angular, Vue.js
  - often include features like component-based architecture, state management, routing, etc.



CDN

- **Content Delivery Network** – distributed servers that deliver content based on user location
- allows loading libraries/frameworks without hosting them locally
- jsDelivr, CDNJS, Google Hosted Libraries, etc.

```html
<script src="https://code.jquery.com/jquery-3.6.1.min.js"
        integrity="sha256-o88AwQnZB+VDvE9tvIXrMQaPlFFSUTR+nldQm1LuPXQ="
        crossorigin="anonymous">
</script>
```

```html
<script src="https://cdn.jsdelivr.net/npm/lodash@4.17.21/lodash.min.js"></script>
```

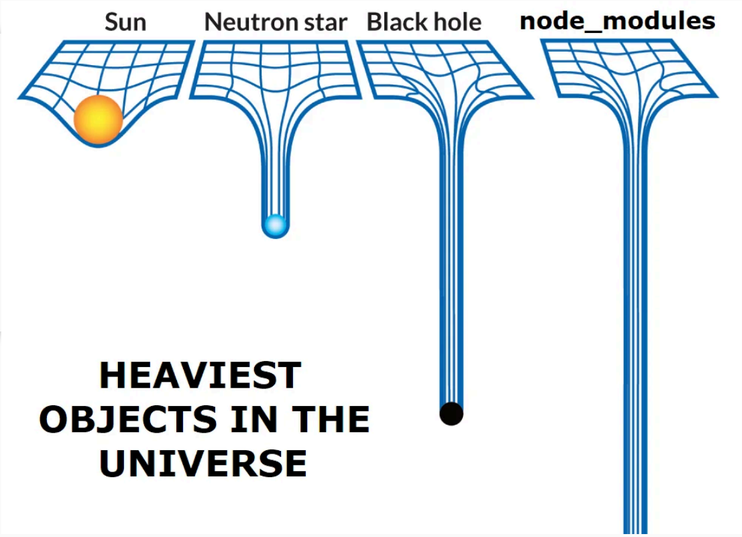
```html
<script src="https://d3js.org/d3.v7.min.js"></script>
```

- .min.js – minified version, smaller file size for faster loading
- integrity – Subresource Integrity (SRI) for security, ensures the file hasn't been tampered with
- crossorigin – CORS settings for SRI, usually set to "anonymous"

npm

- **Node Package Manager** – package manager for JavaScript, used for managing dependencies in projects
- allows installing libraries/frameworks locally in a project

```
# create js project
npm init -y
npm install jquery
npm install lodash
npm install react
```



- creates a package.json file with project metadata and dependencies
- creates a node_modules directory with the installed packages
- can be used with build tools like Webpack, Babel, Vite, etc. for bundling and transpiling code

JQuery

- jQuery, jQuery UI, MIT license
- older popular JavaScript library for DOM manipulation, event handling, and AJAX
- simplifies common tasks and provides cross-browser compatibility

```
<script>
  $(document).ready(function() {
    $("#myButton").click(function() {
      $("ul > li").css("color", "blue");
    });
  });
</script>

<button id="myButton">Winter is Coming</button>

<ul>
  <li class="item">Jon Snow</li>
  <li class="item">Arya Stark</li>
  <li class="item">Tyrion Lannister</li>
</ul>
```

Example

Lodash

- Lodash, MIT license
- utility library for JavaScript, provides functions for common programming tasks
- includes functions for arrays, objects, strings, etc.

```
_.chunk(['a', 'b', 'c', 'd'], 2);
// => [['a', 'b'], ['c', 'd']]
_.shuffle([1, 2, 3, 4]);
// => [4, 1, 3, 2] (example output)
_.merge({ 'a': 1 }, { 'b': 2 }, { 'a': 3 });
// => { 'a': 3, 'b': 2 }
```

D3.js

- D3.js, BSD license, Data Driven Documents
- library for data visualization, allows creating complex and interactive visualizations in the browser
- uses SVG, HTML, and CSS for rendering

```
let data = [ 4, 8, 15, 16, 23, 42 ];

d3.select("body")
  .append("svg")
  .attr("width", 500)
  .attr("height", 500)
  .selectAll("rect")
  .data(data)
  .enter()
  .append("rect")
  .attr("x", (d, i) => i * 80)
  .attr("y", d => 500 - d * 10)
  .attr("width", 40)
  .attr("height", d => d * 10)
  .style("fill", "steelblue");
```

Example

React

- React, MIT license, developed by Facebook
- JavaScript framework for building user interfaces, particularly single-page applications
- uses a component-based architecture, reusable UI components written in JSX (JavaScript XML)
- virtual DOM for efficient updates and rendering

```
import React from 'react';
import ReactDOM from 'react-dom';
import Counter from './Counter'; // another component in a separate file

function App() {
  return (
    <div>
      <h1>Hello, React!</h1>
      <p>This is a simple React component.</p>
      <Counter initialCount={0} />
    </div>
  );
}

ReactDOM.render(<App />, document.getElementById('root'));
```

React

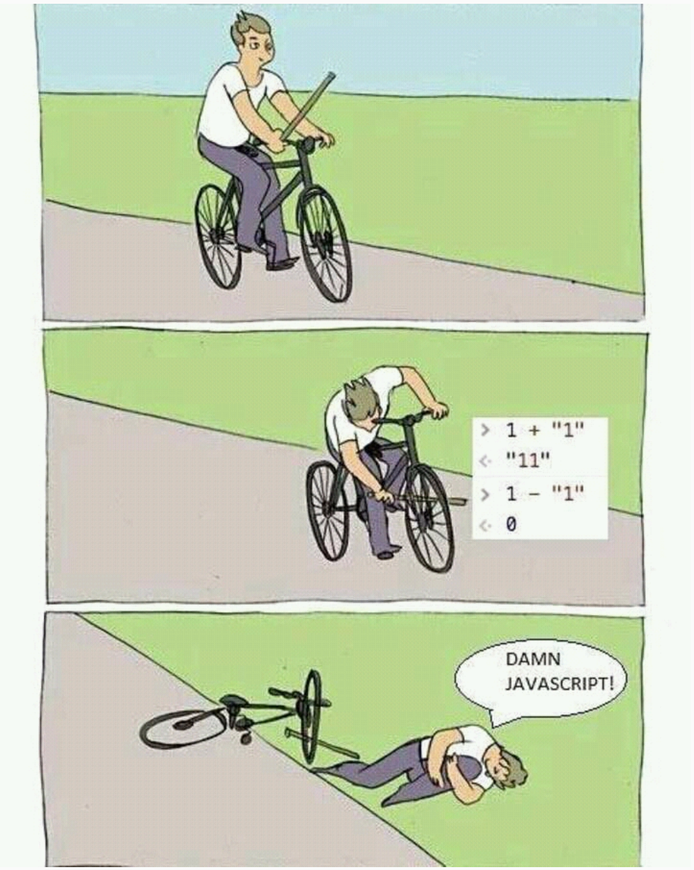
```
// Example of a functional component with state and event handling
import React, { useState } from 'react';

function Counter({ initialCount = 0 }) {
  const [count, setCount] = useState(initialCount);

  return (
    <div>
      <h1>Counter: {count}</h1>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <button onClick={() => setCount(count - 1)}>Decrement</button>
    </div>
  );
}
```

- alternative frameworks: Angular, Vue.js, Svelte, etc.

TypeScript

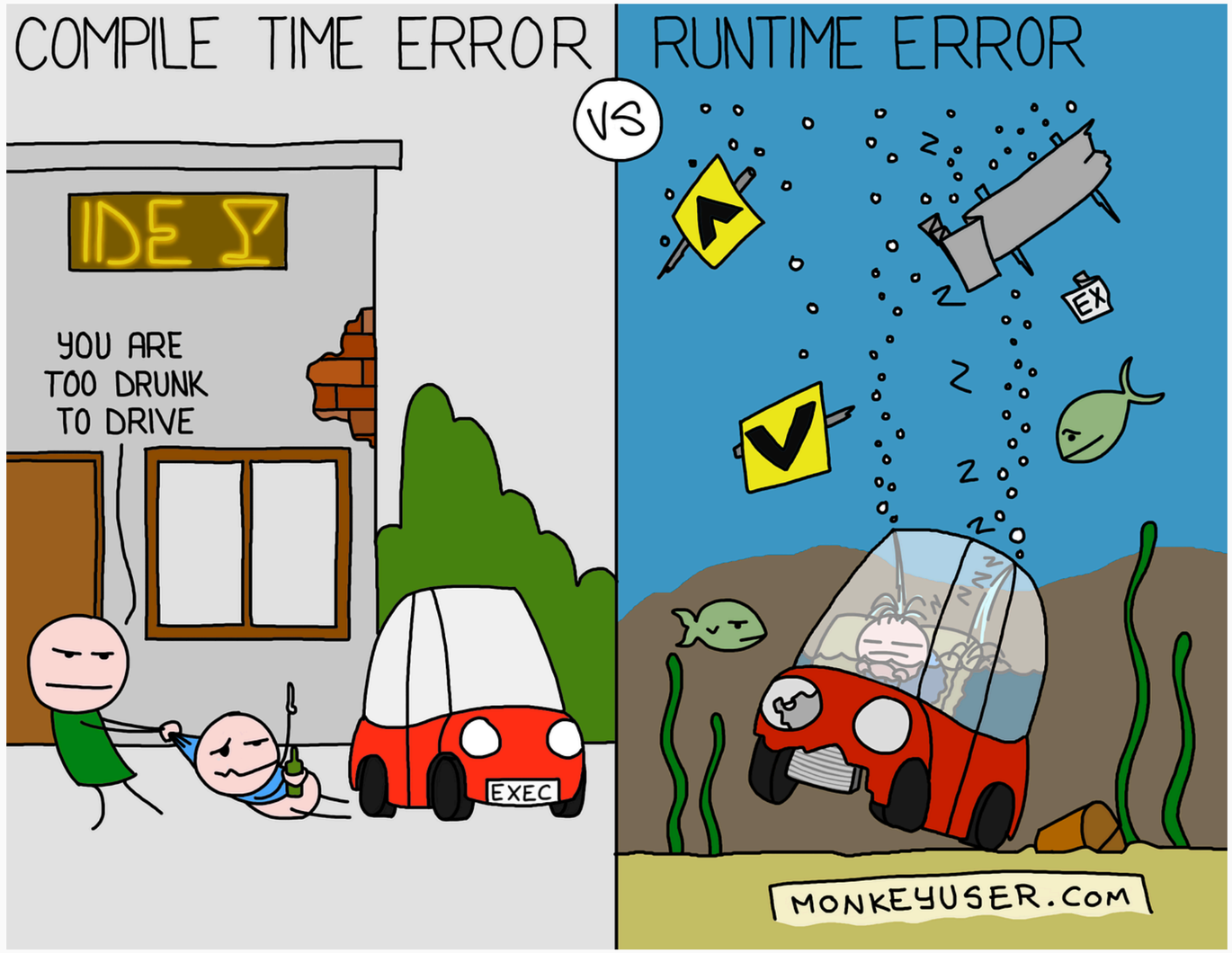
- TypeScript, MIT license, developed by Microsoft
- language, superset of JavaScript that adds static typing and other features
- helps catch errors at compile time and improves code maintainability

```typescript
function add(a: number, b: number): number {
  return a + b;
}

let result: number = add(5, 3);
console.log(result);

 // compile-time error
let result2: number = add("5", "3");
```



- transpiled to JavaScript
  - tsc
  - tools like Webpack, Babel, Vite etc. for bundling large projects

Compile-Time vs Run-Time Error



Summary

links

Key Information

- Ecma International ECMAScript → implemented as JavaScript in browsers
- **Runtime**: engine + call stack + event loop + async (timers, promises)
- **DOM & BOM**: document, window, element manipulation
- **Events**: listeners, mouse/keyboard/touch, DOMContentLoaded, propagation (bubbling/capturing)
- **Networking**: Fetch API → request/response, JSON, HTTP methods
- **Ecosystem**: libraries & frameworks (JQuery, Lodash, D3.js, React, Angular, Vue.js)

To be continued…

HTML – Advanced HTML, XHTML and XML